

PostgreSQL CDC with Debezium

Debezium captures row-level changes in PostgreSQL by reading the write-ahead log (WAL) through PostgreSQL's logical decoding feature, not the table directly. A logical decoding *output plug-in* on the server side turns WAL entries into a consumable stream; Debezium's Java connector reads that stream over the standard replication protocol and emits Kafka events. Unlike Mongo (where the driver already gives you **before/after** as embedded JSON strings), Postgres logical decoding gives you actual structured row data per plug-in.

Two plug-in choices:

- **pgoutput** — built into Postgres 10+, no extra install, maintained by the Postgres community itself (it's the same thing native logical replication uses). Default choice today.
- **decoderbufs** — Debezium-maintained, Protobuf-based, requires installing a separate extension. Mostly legacy at this point; use only if you're stuck on very old Postgres or have a specific reason.

This doc assumes **pgoutput**.

1. PostgreSQL side

1.1 Enable logical decoding

In **postgresql.conf** (requires a server restart):

```
wal_level = logical
max_wal_senders = 4
max_replication_slots = 4
```

- **wal_level = logical** — WAL entries include enough info to reconstruct row changes, not just physical replication. Mandatory; nothing works without it.
- **max_wal_senders** — number of concurrent processes allowed to stream WAL out. One per connector/slot in use, plus headroom.
- **max_replication_slots** — cap on how many slots can exist at once. Each Debezium connector owns exactly one slot.

Check it landed:

```
SHOW wal_level;
```

On managed Postgres (RDS, Cloud SQL, etc.) you usually can't touch **postgresql.conf** directly — there's an instance/parameter-group flag instead (e.g. RDS: **rds.logical_replication = 1**), and it still needs an instance restart to take effect.

1.2 Create a replication user

```
CREATE ROLE MDB_CDC WITH LOGIN PASSWORD '<password>' REPLICATION;  
GRANT SELECT ON ALL TABLES IN SCHEMA mydb TO MDB_CDC;
```

- **REPLICATION** — the role attribute that actually lets the user open a replication connection. Without it Postgres refuses the connector outright, regardless of table grants.
- **SELECT** on the target tables — needed for the initial consistent snapshot. Logical decoding itself doesn't need **SELECT**, but the snapshot phase reads the tables directly.
- If you want Debezium to auto-create the **publication** (see below) rather than doing it yourself, this role also needs **CREATE** on the database, or you pre-create the publication as a superuser and skip that requirement.

1.3 Publication (pgoutput-specific)

pgoutput streams changes for tables that belong to a *publication* — this has no Mongo equivalent, it's a Postgres-native logical replication concept. Debezium creates one automatically on first connect if the user has permission; otherwise create it yourself:

```
CREATE PUBLICATION dbz_publication FOR TABLE mydb.orders, mydb.customers;
```

Match this to the connector's **publication.name** and **table.include.list** — if a table isn't in the publication, Debezium never sees its changes even if it's in **table.include.list**.

1.4 Allow the replication connection

In **pg_hba.conf**, add an entry so the connector's host/user can open a replication connection (not just a regular SQL connection — these are governed separately):

```
host      replication      MDB_CDC      <connector-host-ip>/32      scram-sha-256
```

1.5 Replica identity — the important gotcha

By default (**REPLICA IDENTITY DEFAULT**), **UPDATE/DELETE** events only carry the **primary key** columns in **before** — not the full old row. If your NiFi flow (or anything downstream) needs the complete previous row state — the direct equivalent of Mongo's **capture.mode: change_streams_update_full** — set:

```
ALTER TABLE mydb.orders REPLICA IDENTITY FULL;
```

Without this, don't expect **before** to help you diff old vs. new values; you'll only get the PK.

2. Install the connector on Kafka Connect

Drop the Debezium PostgreSQL connector JAR into your Connect plugin path (e.g. `<kafka-plugins-path>/debezium-connector-postgres/`), restart the Connect workers.

Register via REST API:

```
curl -X POST http://connect-host:8083/connectors \
  -H "Content-Type: application/json" \
  -d @postgres-connector.json
```

`postgres-connector.json`:

```
{
  "name": "postgres-cdc-connector",
  "config": {
    "connector.class":
    "io.debezium.connector.postgresql.PostgresConnector",
    "database.hostname": "<postgres-host>",
    "database.port": "5432",
    "database.user": "MDB_CDC",
    "database.password": "<password>",
    "database.dbname": "mydb",
    "topic.prefix": "dbserver1",
    "table.include.list": "mydb.orders,mydb.customers",
    "plugin.name": "pgoutput",
    "slot.name": "dbz_orders_slot",
    "publication.name": "dbz_publication",
    "snapshot.mode": "initial",
    "tombstones.on.delete": "true",
    "key.converter": "org.apache.kafka.connect.json.JsonConverter",
    "value.converter": "org.apache.kafka.connect.json.JsonConverter",
    "key.converter.schemas.enable": "false",
    "value.converter.schemas.enable": "false"
  }
}
```

What each parameter controls:

- `database.hostname` / `port` / `dbname` — single primary connection, unlike Mongo's multi-host replica-set string. Postgres logical slots only live on a primary; you can't point the connector at a replica.
- `topic.prefix` — same role as in Mongo: prefix for generated topic names, but the naming pattern is `topicPrefix.schemaName.tableName` (schema-qualified), e.g. `dbserver1.mydb.orders` — one topic per table.
- `table.include.list` — whitelist, same idea as Mongo's `collection.include.list`. Fully-qualified `schema.table`.
- `plugin.name` — `pgoutput` or `decoderbufs`. Must match how the server/plugin was set up.
- `slot.name` — the replication slot this connector owns. One connector = one slot, and the slot persists on the server between connector restarts, holding WAL until Debezium confirms it's

processed. **A slot with no connector reading from it is a live WAL-bloat risk** — Postgres won't reclaim WAL segments the slot hasn't confirmed past, even if the connector is gone for good.

- **publication.name** — must match (or be creatable as) the publication from step 1.3.
- **snapshot.mode** — same menu of values as Mongo conceptually (**initial**, **no_data**, **never/initial_only** etc.), plus a few Postgres-only modes (**always**, **when_needed**, **configuration_based**) for things like recovering after a failover promotes a new primary.
- **tombstones.on.delete** — same as Mongo: null-value marker after deletes for Kafka log compaction, filtered out in NiFi.

Check it came up:

```
curl http://connect-host:8083/connectors/postgres-cdc-connector/status
```

3. Event shape

```
{
  "before": null,
  "after": {
    "id": 1,
    "status": "shipped",
    "amount": 250
  },
  "op": "u",
  "ts_ms": 1752480000123
}
```

Key difference from Mongo: **after** (and **before**) are already structured JSON objects, not JSON-encoded strings. No double-parsing needed in NiFi — one **EvaluateJsonPath** pass gets you there.

op values (Postgres has two more than Mongo):

- **r** — snapshot read (initial load, treat as insert)
- **c** — insert
- **u** — update
- **d** — delete
- **t** — truncate (whole table wiped; no message key, needs its own handling — see below)
- **m** — generic logical decoding message (only via **pgoutput** on Postgres 14+, rare unless something explicitly calls **pg_logical_emit_message**)

before on **u/d** events only contains what **REPLICA IDENTITY** exposes — PK columns only by default, full row if you set **REPLICA IDENTITY FULL** (see 1.5).

Also note: a primary-key **update** doesn't come through as a single **u** event — Debezium splits it into a **d** (old key) + **c** (new key), each carrying a message header (**__debezium.newkey** / **__debezium.oldkey**). If your NiFi flow assumes PK is immutable per row, this is the edge case that breaks that assumption.